Demo app: PharmacyApi — .NET 8 Web API (C#)

AI-Assisted Development, Code Review & Test Generation

Using AI as a Force Multiplier — Not a Replacement for Judgment

What We'll Cover

1. Where AI coding assistants fit in the development lifecycle
2. Where AI genuinely helps — and where it falls short
3. AI-assisted code review: a first pass, not a replacement for one
4. AI-assisted test generation: coverage vs. meaningful coverage
5. Prompting discipline: context, constraints, and verification
6. Trust boundaries: what to verify vs. what to accept outright
7. Responsible use: privacy, IP, and team workflow guidelines
8. Hands-on: reviewing and testing a real C# change with AI assistance

Learning Objectives

- Identify which development tasks AI coding assistants genuinely speed up, and which ones still require human judgment
- Use an AI assistant to generate a first-pass code review on a C# pull request
- Use an AI assistant to generate xUnit tests for a .NET method, and evaluate whether the generated assertions are meaningful
- Distinguish test coverage as a quantity metric from test quality as a correctness measure
- Write prompts for code tasks that supply real context, explicit constraints, and a verification step
- Apply a risk-based framework to decide how much scrutiny AI-generated code needs before it merges
- Recognize the privacy and intellectual-property considerations that come with using AI coding tools

PART 1

AI in the Software Development Lifecycle

The Rise of AI Coding Assistants

- AI coding tools have moved from novelty to daily use across most professional development teams within a few short years
- They now touch nearly every stage: drafting code, reviewing it, writing tests for it, explaining it, and documenting it
- The tools keep changing quickly — the durable skill isn't memorizing one tool's interface, it's learning how to use any of them well and safely
- Using these tools well is now a professional skill in its own right, distinct from simply knowing how to write code

This module's focus

Not which tool to buy — how to use AI assistance for code review and test generation in a way that holds up under scrutiny.

Types of AI Coding Tools

- Inline completion — suggests the next few lines as you type, directly inside the editor (for example, GitHub Copilot's inline suggestions)
- Chat-based assistants — a conversational interface where you describe a task or paste code and get a response back, often inside the IDE
- Agentic tools — given a goal, can read multiple files, make a plan, and carry out a multi-step change across a codebase with less direct supervision
- Review bots — run automatically against a pull request and post comments, the same way a linter or static analyzer would
- Test generators — given a method or class, produce a starting set of unit tests, often as a feature inside a chat-based or agentic tool
- This module focuses mainly on the review-bot and test-generator use cases, since those are the ones this course's pipeline work depends on

Where AI Genuinely Helps — Drafting & Boilerplate

- Scaffolding a new controller, DTO, or repository class that follows a pattern already established elsewhere in the codebase
- Writing mapping code between a database entity and a response model — repetitive, mechanical, easy to verify by eye
- Generating a first draft of a configuration file, a Dockerfile, or a pipeline YAML stage, to then be reviewed and adjusted
- Converting a data shape from one format to another — a JSON sample into a C# record, for instance

In PharmacyApi

Asking an assistant to draft a PrescriptionDto record and its mapping from the Prescription entity saves real typing — and it's trivially easy to check by eye.

Where AI Genuinely Helps — Review & Explanation

- A fast first pass over a pull request: flagging an obvious null-reference risk, an inconsistent naming pattern, a missing error handler
- Summarizing a large, unfamiliar diff before a human reviewer dives in — orienting them before they spend their own attention
- Explaining a block of unfamiliar code — what it does, what inputs it expects, what could go wrong
- Suggesting edge cases a reviewer might not immediately think to check, as prompts for human judgment rather than verdicts

The key phrase

"First pass." Every one of these items ends with a human still reading the result and deciding what to do with it.

Where AI Genuinely Helps — Test Generation

- Producing a starting set of xUnit tests for an existing method — faster than writing the test scaffolding from a blank file
- Suggesting edge cases for a method's inputs — a null argument, an empty collection, a boundary value — that are easy to miss when writing tests by hand
- Generating tests for a batch of similar methods that all follow the same shape, once a human has reviewed the first one
- Filling test coverage gaps identified by a coverage report, as a starting point for a human to refine

The catch

A generated test is a draft. Whether it actually verifies correct behavior — not just that the code ran — still needs a human to check. More on this later in the module.

Where AI Falls Short — Business Logic & Domain Knowledge

- AI doesn't know your domain rules unless you explicitly supply them — it can't infer that a prescription refill rule depends on a regulation it was never told about
- It can write code that compiles and looks reasonable while being completely wrong about what the business actually needs
- It has no memory of a decision your team made in a meeting last month, or a trade-off that was deliberately chosen for reasons not written down anywhere
- Verifying business correctness is, and remains, a human responsibility — no amount of context in a single prompt fully substitutes for domain expertise

Example

An assistant can write a syntactically perfect dosage-calculation method that silently uses the wrong rounding rule for a controlled substance — and it will look completely confident doing it.

Where AI Falls Short — Architecture & Accountability

- Choosing between two valid architectural approaches requires weighing trade-offs specific to your team, your timeline, and your constraints — context an AI tool doesn't have
- An AI review tool only catches what it was scoped to look for — it has no sense of the parts of a change that matter most and were never flagged
- A pull request still needs a named human approver — accountability for a merged change can't be delegated to a tool
- “The AI said it was fine” is never an acceptable answer to “why did this ship broken”

Teaching point

Ask the room: who is accountable when an AI-reviewed, AI-tested change breaks production? The answer is always the human who approved it — never the tool.

The Hallucination Risk

- AI tools can generate plausible-sounding but entirely incorrect information — a NuGet package that doesn't exist, a method signature that was never real, an API behavior that was invented
- Generated code can compile and even run while still being subtly wrong — confidence in the tool's tone is not evidence of correctness
- A hallucinated package name in a dependency file is a particular risk: if an attacker has registered that exact name, you may pull in malicious code without realizing it
- The defense is always the same: verify. Run it, test it, read the diff, check that referenced APIs and packages actually exist before trusting the output
- This risk doesn't disappear as tools improve — it just becomes less frequent, which can make it more dangerous because trust builds faster than vigilance does

Knowledge Check

An AI assistant suggests adding a NuGet package to fix a compile error. The package name looks plausible but you don't recognize it. What should you do first?

A. Add it immediately — the assistant clearly knows the ecosystem better than you do

B. Verify the package actually exists and is the official, trusted source before adding it to the project

C. Ignore the suggestion entirely and never use AI for dependency issues again

D. Add it, but only to a test project, never to production code

Answer

B — Correct

A hallucinated or typosquatted package name is one of the more dangerous failure modes of AI assistance. Confirming it exists and is the legitimate source takes seconds and closes off a real supply-chain risk.

- A is wrong — confidence in a suggestion's tone is not evidence that it's correct or safe
- C is wrong — the fix isn't to stop using the tool, it's to verify its output, exactly like any other suggestion from any other source
- D is wrong — a malicious package in a test project can still compromise a build environment

PART 2

AI-Assisted Code Review

What an AI Code Review Actually Does

- Runs against a pull request the same way a linter or static analyzer does — as an automated pass that posts comments before a human opens the diff
- Reads the changed files (and often surrounding context) and compares what it sees against common patterns, known bug shapes, and whatever instructions it was given
- Posts specific, line-level comments — not a single pass/fail verdict for the whole pull request
- Can be scoped: told to focus on security, on style consistency, on null-safety, or left general-purpose

In one sentence

It's a second set of eyes that never gets tired and never skips a file — not a second decision-maker.

What AI Reviewers Catch Well

- A possible NullReferenceException — a property accessed without a prior null check on a value that could legitimately be null
- Inconsistent naming — a new method that breaks an established PascalCase or async-suffix convention elsewhere in the codebase
- A missing try/catch, or a catch block that swallows an exception without logging it
- An obvious off-by-one error in a loop boundary, or a condition that looks inverted from what the surrounding code implies it should be
- A resource (a DbContext, a stream) that isn't properly disposed

Pattern, not judgment

Every item here is a recognizable pattern, not a business decision. That's exactly the category AI review tools are strongest at.

Code: A C# Method With Issues

- A method from a PharmacyApi pull request, before review — look for what an AI reviewer (and a human one) should flag

```
public async Task<PrescriptionDto> GetPrescription(int id)
{
    var prescription = await _db.Prescriptions
        .FirstOrDefaultAsync(p => p.Id == id);

    return new PrescriptionDto
    {
        DrugName = prescription.DrugName,
        Dosage = prescription.Dosage,
        PrescribedBy = prescription.Doctor.FullName
    };
}
```

Code: AI Review Comments on That Method

- What an AI reviewer flags on the method from the previous slide — each comment anchors to a specific line

```
// Line 4: FirstOrDefaultAsync can return null.
// prescription.DrugName below will throw a
// NullReferenceException if no match is found.
// Suggest: check for null and return 404 Not Found.

// Line 10: prescription.Doctor could be null if the
// relationship isn't eager-loaded (.Include(p => p.Doctor)).
// This will throw at run time under real data, not just
// in the not-found case above.

// No [HttpGet] / route attribute visible in this excerpt
// — confirm this method is wired up as expected.
```

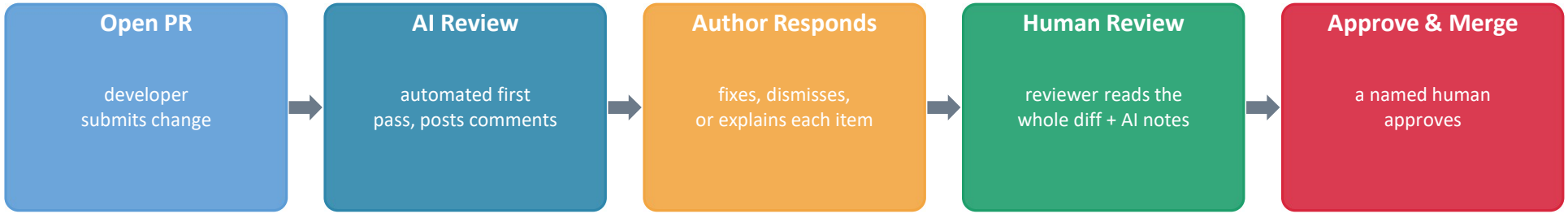
What AI Reviewers Miss

- Whether the chosen approach is the right one for this feature — a valid, bug-free implementation of the wrong design still ships the wrong thing
- Whether a trade-off the team already discussed and deliberately accepted was correctly implemented, when that context was never given to the tool
- Domain-specific correctness — whether a dosage calculation follows the actual clinical rule, not just whether the arithmetic is internally consistent
- Anything outside the scope it was told to check — an AI review focused on null-safety has nothing useful to say about an authorization bug unless it was asked to look for one

Teaching point

A clean AI review means “no issues within what it was asked to check” — never “this change is correct.” The distinction matters every single time.

Human-in-the-Loop: The Review Workflow



- The AI pass runs before a human spends their attention — it narrows what the human needs to think hardest about, it doesn't replace their review
- The final approve step always belongs to a human, regardless of how clean the AI pass came back

Scoping an AI Review

- An AI review tool is only as useful as the instructions it's given — a vague “review this” produces a vague, generic pass
- Specify what matters for this change: “focus on null-safety and exception handling in the data access layer” produces sharper, more relevant findings than a blanket request
- Give it the same context a human reviewer would want: the related interface, the test file, the ticket describing what the change is supposed to do
- Different changes deserve different scopes — a security-sensitive change to authentication code warrants an explicit security-focused pass, not just the default general review
- Re-scoping as needed is a normal part of the workflow, not a sign the tool isn't working

Knowledge Check

An AI code review tool approves a pull request with no comments. What should that mean to the team?

A. The PR is safe to merge without a human reviewer looking at it

B. The AI found nothing worth flagging within what it was asked to check — a human reviewer still needs to approve it

C. The code has 100% test coverage

D. The code definitely satisfies the business requirements

Answer

B — Correct

A clean AI pass means no issues in that scope — it says nothing about business correctness, architecture fit, or anything outside what it was asked to look for.

- A is wrong — a PR still needs a human approver regardless of what an automated first pass found
- C is wrong — code review and test coverage are two separate, unrelated measurements
- D is wrong — AI has no way to verify business requirements it was never given

PART 3

AI-Assisted Test Generation

Unit Testing in .NET: A Quick Recap

- xUnit, NUnit and MSTest are the three mainstream .NET testing frameworks — xUnit is the most common default for new projects
- A unit test exercises one unit of behavior in isolation — typically one method, with its dependencies replaced by mocks or fakes
- The Arrange-Act-Assert pattern structures most tests: set up the inputs, call the method, then check the output matches what's expected
- A test suite's value comes from what it verifies, not from how many of them exist — this distinction becomes central once AI starts generating tests

In PharmacyApi

Tests use xUnit with Moq for mocking the database context, following Arrange-Act-Assert throughout the test project.

Generating Tests With AI — the Starting Point

- Point an assistant at an existing method and ask it to generate xUnit tests — it produces a test class with several test methods in seconds
- This is genuinely useful for getting past a blank file, and for covering the straightforward happy-path cases quickly
- What it tends to do well by default: cover the typical, expected input and confirm the method returns roughly the expected shape of result
- What it tends to miss without explicit prompting: null inputs, empty collections, boundary values, concurrent access, and failure paths
- Treat the output as a first draft a human reviews and extends — exactly the same posture as AI-assisted code review

Code: A C# Service Method to Test

- A PharmacyApi service method we'll ask an assistant to generate xUnit tests for

```
public class RefillService
{
    public RefillResult CalculateRefillsRemaining(
        Prescription prescription, int refillsUsed)
    {
        if (refillsUsed > prescription.MaxRefills)
        {
            return RefillResult.Exceeded;
        }

        var remaining =
            prescription.MaxRefills - refillsUsed;

        return remaining == 0
            ? RefillResult.FinalRefillUsed
            : RefillResult.Available(remaining);
    }
}
```

Code: AI-Generated xUnit Test

- A generated test for CalculateRefillsRemaining — worth reading closely before trusting it

```
[Fact]
public void CalculateRefillsRemaining_ReturnsResult()
{
    var prescription = new Prescription { MaxRefills = 3 };
    var service = new RefillService();

    var result = service.CalculateRefillsRemaining(
        prescription, refillsUsed: 1);

    Assert.NotNull(result);
}

// Runs without throwing — but does it verify the
// actual refill count, or just that something came back?
```

Coverage vs. Quality: The Metric Trap

- Code coverage measures which lines executed while tests ran — it says nothing about whether the right outcome was verified
- A test that calls a method and asserts only Assert.NotNull(result) raises the coverage number without raising actual confidence in correctness
- A suite can report 90% coverage and still miss the one bug that actually matters, if the assertions never check the specific value that would reveal it
- Coverage is a useful signal for what's untested — it is not, by itself, evidence that what is tested is tested well

Rule of thumb

If you could change the method's return value to something wrong and every test would still pass, the tests aren't actually testing behavior.

Reading a Generated Test Critically

- The same test, strengthened — compare what each assertion actually verifies against the weaker version two slides back

```
[Fact]
public void CalculateRefillsRemaining_WithOneUsed_
    ReturnsTwoRemaining()
{
    var prescription = new Prescription { MaxRefills = 3 };
    var service = new RefillService();

    var result = service.CalculateRefillsRemaining(
        prescription, refillsUsed: 1);

    Assert.Equal(RefillResult.Available(2), result);
}
```

Edge Cases AI Tends to Miss

- refillsUsed exactly equal to MaxRefills — the boundary where the method should report the final refill, not just “available” or “exceeded”
- refillsUsed greater than MaxRefills — confirming the Exceeded path actually triggers, not just that the Available path works
- A negative refillsUsed value — does the method handle invalid input gracefully, or does it silently produce a nonsensical result?
- A null prescription argument — does the method throw the exception type callers expect, or something less predictable?
- Without being explicitly asked to consider boundaries and invalid input, a generated test suite tends to cluster around the single obvious happy-path case

Prompting for Better Test Coverage

- Compare a vague test-generation request against a specific one — same method, very different result

```
Vague:
  "Write tests for CalculateRefillsRemaining."

Specific:
  "Write xUnit tests for CalculateRefillsRemaining
   covering: refillsUsed below MaxRefills, exactly
   equal to MaxRefills, and greater than MaxRefills.
   Assert on the actual RefillResult value returned,
   not just that a result is non-null."
```

Knowledge Check

AI generates a unit test that calls a service method and asserts only that no exception was thrown — it doesn't check the returned value. Test coverage goes up. Has test quality improved?

A. Yes, higher coverage always means better-tested code

B. Not necessarily — the test exercises the line but doesn't verify correct behavior, so a wrong return value would still pass it

C. No, because generated tests never run in a build pipeline

D. Yes, because any assertion at all makes a test meaningful

Answer

B — Correct

A test with no assertion on the actual output will pass even if the method returns the wrong value — it only proves the code didn't crash.

- A is wrong — coverage is a quantity metric, not a quality one; this is the whole point of the slide
- C is wrong — generated tests run in a build pipeline exactly like any other test once committed
- D is wrong — an assertion only adds value if it checks something meaningful about correctness

PART 4

Prompting Discipline & Trust Boundaries

Prompting Fundamentals: Context, Constraints, Verification

- Context — paste the actual file, the actual interface it implements, the actual error message, not a paraphrase of what the code does
- Constraints — name the framework and version (.NET 8, xUnit), the existing patterns in the codebase, and what must not change
- Verification — every answer needs a next step that checks it: run the tests, run the build, read the diff line by line before it's trusted
- A vague prompt gets a vague, possibly wrong answer; a specific prompt with real constraints gets a usable one

The three together

Context without verification is how a wrong answer gets merged with confidence. Verification without context is how you waste time re-explaining what the tool should already know.

Prompt Examples: Vague vs. Specific

- The same request to fix a bug, written two different ways

```
Vague:
  "Fix this."

Specific:
  "This GET /prescriptions/{id} endpoint throws a
   NullReferenceException when the prescription has
   no associated Doctor. We're on .NET 8 / EF Core 8.
   Add a null check and return 404 NotFound using the
   same pattern as GetPatient() in PatientController.cs.
   Don't change the response DTO shape."
```

Common Prompting Mistakes

- Describing the code from memory instead of pasting it — small inaccuracies in the description become the basis for a wrong answer
- Asking for a fix without stating the constraint that matters most — “don't change the public API” left unsaid, then discovering the suggested fix broke every caller
- Accepting the first answer without a verification step — treating a confident-sounding response as equivalent to a correct one
- Asking one enormous, multi-part question instead of breaking a complex task into smaller, checkable steps
- Omitting the framework or version — a suggestion written for an older .NET version can compile cleanly while using an approach the team has since moved away from

Trust Boundaries: A Risk-Based Framework

- Not every piece of AI-generated code deserves the same level of scrutiny — the right amount of checking scales with what's actually at stake
- Low stakes, easy to verify — reasonable to accept quickly after a glance
- Medium stakes — business logic, a new endpoint, a bug fix — always run it, always read the diff, never merge on trust alone
- High stakes — authentication, payments, anything touching patient data, security-sensitive code — treat it with the same scrutiny as if a brand-new engineer had written it
- The gates that already exist in a delivery pipeline — tests, linting, security scanning — don't go away because AI wrote the code; every line still has to pass them

Low / Medium / High Stakes — C# Examples

- Low stakes — a DTO's property list, a getter-only record, a logging message's wording
- Medium stakes — the refill-calculation logic in RefillService, a new validation rule on an incoming request, a change to an EF Core query's filter
- High stakes — the authentication middleware, the authorization check on a controlled-substance endpoint, any code handling a patient's personal health information
- The category isn't about how complex the code looks — a one-line change to an authorization check is high stakes; a fifty-line DTO is still low stakes

AI-Authored Code Still Needs the Same Gates

- A build pipeline doesn't know or care who or what wrote a line of code — it runs the same unit tests, the same linter, the same security scans regardless
- Treating an AI-authored pull request as lower-risk because it “looks clean” is a mistake — clean-looking code and correct code are not the same thing
- A dependency an assistant suggested still goes through the same vulnerability scan as any other package added by a human
- The review process itself doesn't shrink — a human approver is still required on every pull request, AI-assisted or not

Responsible Use: Data Privacy & Intellectual Property

- Some AI coding tools may retain prompts and the code pasted into them, depending on the product and its configuration — check your organization's approved tools and their data-handling terms before pasting proprietary or sensitive code
- Code generated by an AI tool can resemble existing licensed code it was trained on — understand your organization's policy on reviewing generated code for licensing concerns before shipping it
- Client contracts may place specific restrictions on which tools can be used with their codebase or data — these apply to AI assistants exactly as they apply to any other third-party tool
- When in doubt about what's permitted, ask — this is a policy question, not a technical one, and the answer varies by organization and client

A good habit

Treat an AI coding tool the same way you'd treat any other third-party service that could see your code — know what it does with what you send it.

Avoiding Leaked Secrets in Prompts

- Pasting a real connection string, API key, or customer record into a prompt “for context” sends that data outside your organization's own systems
- This is the same category of risk as committing a secret to source control — once sent, you can't fully take it back
- Use placeholder values when sharing a config file or an error message for help — the shape of the problem is usually enough context without the real values
- If a real secret is ever accidentally pasted into a prompt, treat it exactly like a leaked credential: rotate it at the source
- The same discipline that keeps secrets out of commits and logs applies here too — it's just a new place the old habit needs to extend to

Knowledge Check

A developer pastes a production database connection string into an AI chat tool to help debug a connection error. What's the right way to think about this?

A. It's fine, since the chat conversation is private to that one developer

B. The same risk as committing the secret to source control — it should be treated as potentially exposed, and the credential should be rotated

C. It's only a problem if the chat tool is free rather than a paid product

D. It's fine as long as the connection string is deleted from the chat afterward

Answer

B — Correct

Whether a tool retains the data or not, the moment a real credential leaves your organization's controlled boundary, the safest assumption is that it's been seen somewhere it shouldn't have been — rotate it.

- A is wrong — “private to me” doesn't mean the data wasn't transmitted to and processed by a third-party system
- C is wrong — pricing tier has no bearing on data handling risk; always check the tool's actual data policy
- D is wrong — deleting a message from a chat interface doesn't guarantee the data was never logged, cached, or processed upstream

Applying This: A Hands-On Walkthrough

Generate a review and a test suite — then verify both by hand

- Take a real PharmacyApi change with a deliberately introduced bug and ask an AI assistant to review it
- Check which findings are real, which are noise, and what a human reviewer still has to catch on top of them
- Ask the assistant to generate xUnit tests for RefillService, including edge cases around the refill boundary
- Read each generated test and decide: does this actually assert correct behavior, or just that nothing threw?

Full step-by-step instructions are in the separate Labs document — this slide is the "why," not the "how."

Common Pitfalls & Anti-Patterns

- Merging an AI-authored or AI-reviewed pull request faster than a human-authored one, simply because it “looks clean”
- Treating a clean AI review as proof of business correctness, rather than proof that nothing in its given scope was flagged
- Accepting a generated test suite's coverage number without reading whether the assertions check anything meaningful
- Pasting real secrets, customer data, or proprietary algorithms into a prompt without checking the tool's data-handling policy first
- Letting a complex architectural decision get settled by what an AI assistant suggested, instead of by the team that owns the trade-off
- Skipping the human approver step on a pull request because “the AI already reviewed it”

Best Practices Checklist

- Every pull request still has a named human approver, regardless of how clean an AI review pass came back
- AI review scope is stated explicitly — what it was asked to check, so everyone reading its comments knows what it wasn't asked to check
- Generated tests are read line by line for what they actually assert, not just trusted for the coverage number they produce
- Prompts for code tasks include real context, explicit constraints, and end with a concrete verification step
- Scrutiny on AI-generated code scales with what's actually at stake — low, medium, and high-stakes changes are not reviewed the same way
- No real secret, customer data, or proprietary logic is pasted into a prompt without first checking the tool's data policy

Key Takeaways

- AI is a strong first-pass reviewer and a fast way to draft tests — it is not a replacement for human judgment on correctness or design
- Every pull request still needs a human approver; every AI finding is input to that decision, not the decision itself
- Coverage percentage and test quality are different things — read what a generated test actually asserts before trusting it
- Good prompts for code tasks give real context, explicit constraints, and a verification step — vague prompts get vague, risky answers
- Trust scales with stakes — low-stakes code earns a glance, high-stakes code earns the same scrutiny as if a new engineer wrote it
- The existing quality and security gates in a pipeline don't relax because AI wrote the code — every line passes the same checks either way

PharmacyApi — .NET 8 Web API (C#)

Thank You

Questions & Discussion